

Advanced Natural Language Processing

Discussion 09: Fine-Tuning, RLHF, Reasoning

Department of Advanced Computer Sciences (DACS)
Master of AI / DSDM
Prof dr ir Jan C. Scholtes – Course year 2026-2027



Maastricht University

Pre-Training & Fine-Tuning



Maastricht University

Pre-training and Fine-Tuning BERT

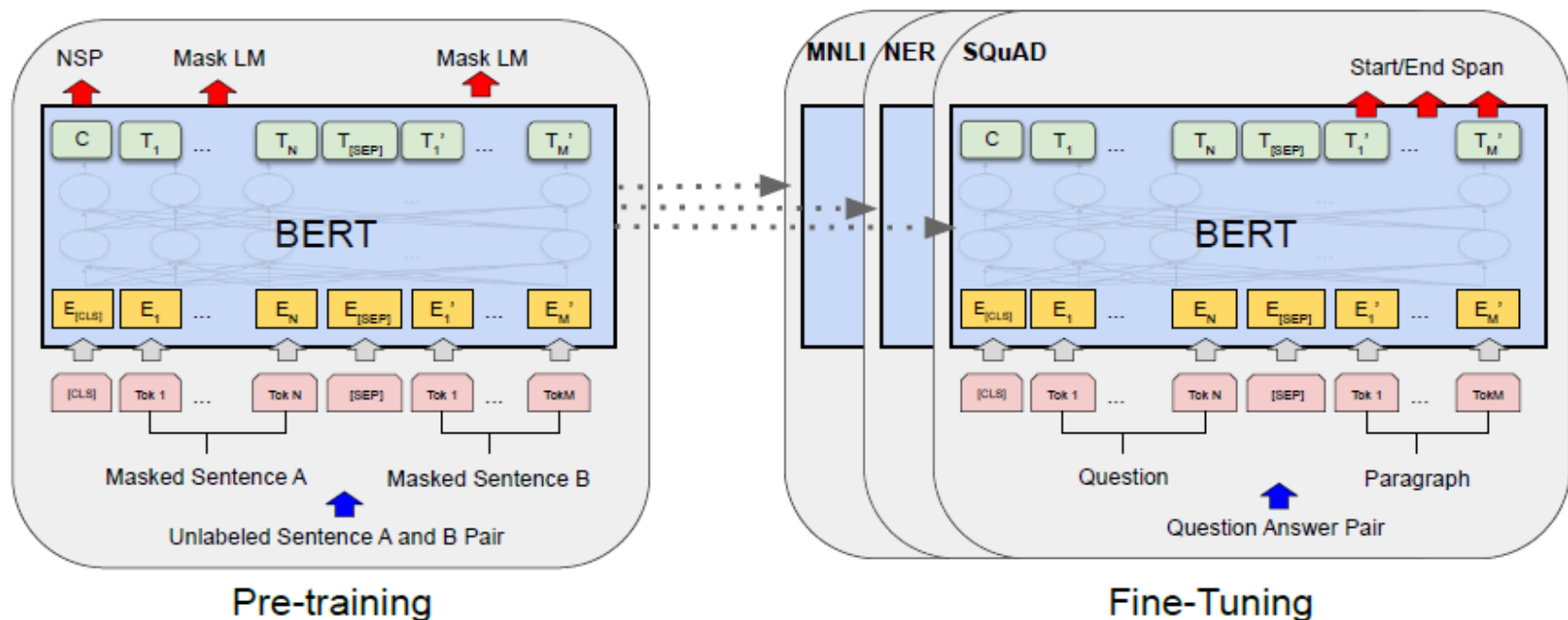


Figure 1: Overall pre-training and fine-tuning procedures for BERT. Apart from output layers, the same architectures are used in both pre-training and fine-tuning. The same pre-trained model parameters are used to initialize models for different down-stream tasks. During fine-tuning, all parameters are fine-tuned. [CLS] is a special symbol added in front of every input example, and [SEP] is a special separator token (e.g. separating questions/answers).

BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding, Jacob Devlin et al., May 2019.
<https://arxiv.org/pdf/1810.04805.pdf>

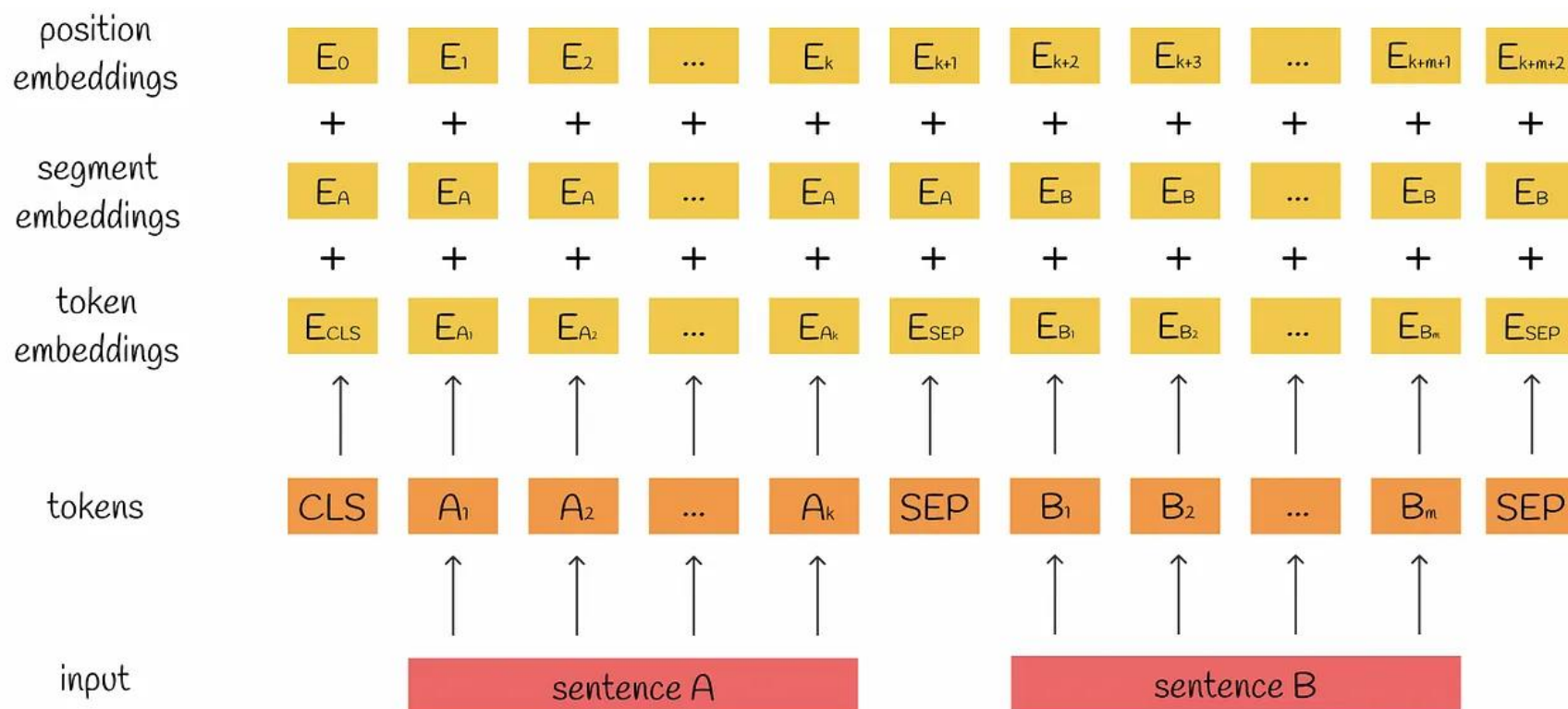
Pre-training and fine-tuning BERT: input

- For the input, BERT takes a single sequence or a pair of sequences. Each sequence is split into tokens.
- Additionally, two special tokens are passed to the input:
 - [CLS] — passed before the first sequence indicating its beginning: CLEAR SENTENCE. At the same time, [CLS] is also used for a classification objective during training. [SEP] — passed between sequences to indicate the end of the first sequence and the beginning of the second: SEPARATOR.
- Sending two sequences makes it possible for BERT to handle a variety of tasks where an input contains a pair of sequences (e.g. question and answer, hypothesis and premise, etc.).

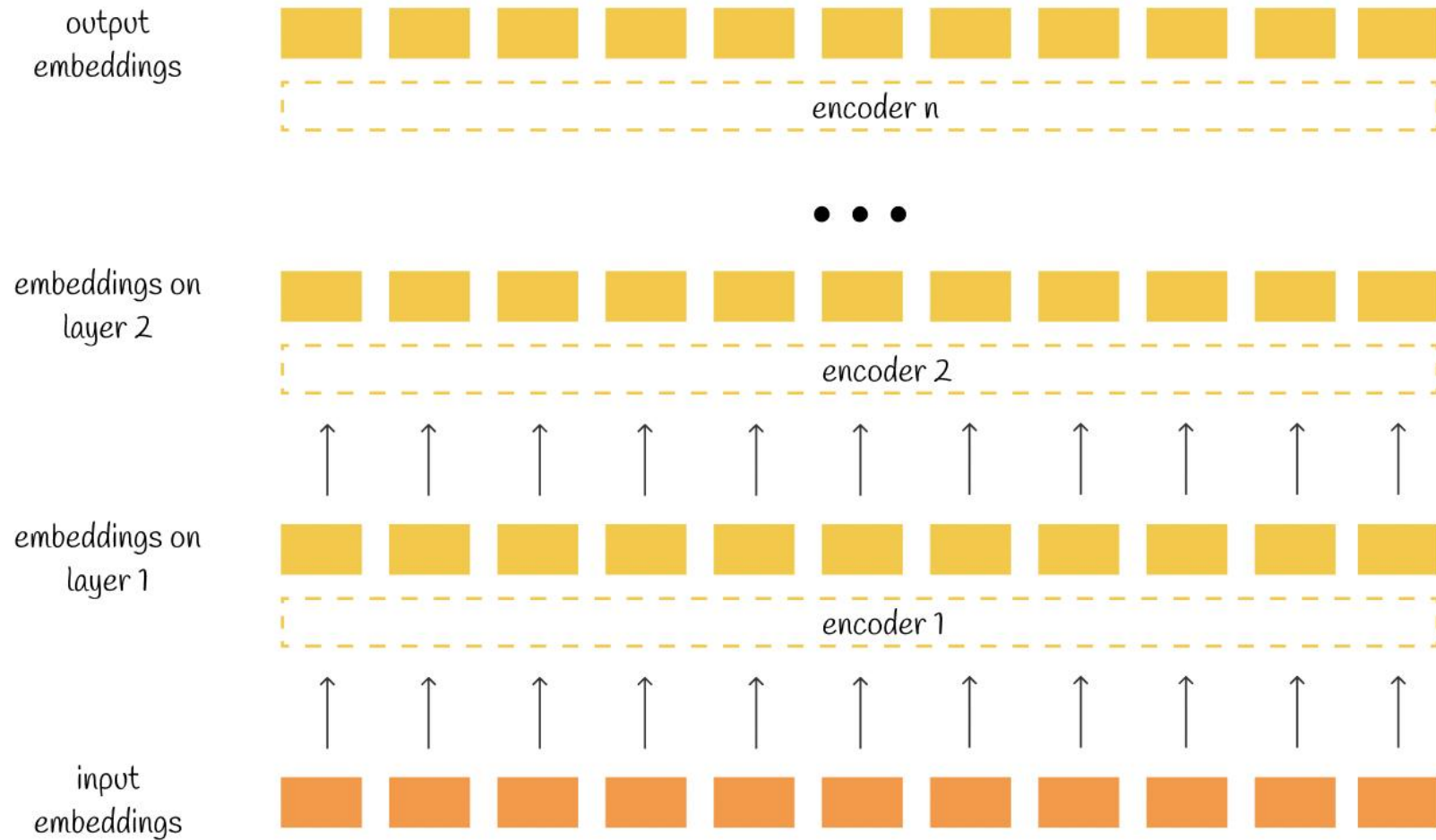


Pre-training and fine-tuning BERT: Embeddings

After tokenization, an embedding is built for each token. To make input embeddings more representative, BERT constructs three types of embeddings for each token: **Token embeddings**, **segment embeddings** and **positional embeddings**.



Pre-training and Fine-Tuning BERT: Layers



Pre-Training and Fine-Tuning BERT

- **Pre-training:** BERT is pre-trained on unlabeled pair of sequences over two prediction tasks:
 - masked language modeling (MLM)
 - natural language inference (NLI).

We will discuss in detail what this is in the subsequent slides. For each pair of sequences, the model makes predictions for these two tasks and based on the loss values, it performs backpropagation to update weights.

- **Fine-tuning:** BERT is initialized with pre-trained weights during the above pre-training and can then be fine-tuned for a particular problem on labeled data during the fine-tuning process.



BERT: Training Details we will discuss...

- BERT is pre-trained on BooksCorpus (800M words) and English Wikipedia (2,500M words). For extracting longer continuous texts, authors took from Wikipedia only reading passages ignoring tables, headers and lists.
- BERT is trained on a million batches of size equal to 256 sequences which is equivalent to 40 epochs on 3.3 billion words. Each sequence contains up to 128 (90% of the time) or 512 (10% of the time) tokens.
- The training parameters are the following:
 - Optimizator: Adam (learning rate $\ell = 1e-4$, weight decay $L_2 = 0.01$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 1e-6$).
 - Learning rate warmup is performed over the first 10.000 steps and then reduced linearly.
 - Dropout ($\alpha = 0.1$) layer is used on all layers.
 - Activation function: GELU.
 - Training loss is the sum of mean MLM and mean next sentence prediction likelihoods.

BERT: Masked-Language Modeling (MLM)

1. 15% of tokens are randomly masked. The masked tokens will be then predicted at the end of the iteration.

2. The masked tokens are replaced in one of three ways:

- -80% of the tokens are replaced by the **[MASK]** token.
 - Example: I bought a book → I bought a [MASK]
- -10% of the tokens are replaced by a random token.
 - Example: He is eating a fruit → He is drawing a fruit
- 10% of the tokens remain unchanged.
 - Example: A house is near me → A house is near me

3. All tokens are passed to the BERT model which outputs an embedding for each token it received as input.

4. Output embeddings corresponding to the tokens processed at step 2 are independently used to predict the masked tokens. The result of each prediction is a probability distribution across all the tokens in the vocabulary.

5. The cross-entropy loss is calculated by comparing probability distributions with the true masked tokens.

6. The model weights are updated by using backpropagation.

Masked Language Modeling

Masked language modeling is a fill-in-the-blank task, where a model uses the context words surrounding a [MASK] token to try to predict what the [MASK] word should be.

The model shown here is **BERT**, the first large transformer to be trained on this task. Enter text with one or more "[MASK]" tokens and the model will generate the most likely substitution for each.

Sentence:

It is [MASK] to
[MASK] that

Mask 1 Predictions:

11.5% **important**
11.1% **difficult**
8.8% **easy**
5.0% **possible**
3.5% **hard**

Mask 2 Predictions:

22.9% **say**
15.5% **note**
5.7% **see**
3.3% **suggest**
2.1% **conclude**

BERT: Natural Language Inference (NLI)

- BERT learns how to predict the second sequence follows the first. The prediction is made by using only the embedding from the final hidden state of the [CLS] token which is supposed to contain aggregated information from both sequences: **Question: How is this represented?**
- Similarly to Mask Language Modeling, a constructed probability distribution (binary in this case) is used to calculate the model's loss and update the weights of the model through backpropagation.
- For NLI, in the original BERT paper, it is recommended choosing 50% of pairs of sequences which follow each other in the corpus (positive pairs) and 50% of pairs where sequences are taken randomly from the corpus (negative pairs).



We will get back to this strategy later in the section on
“contrastive learning”

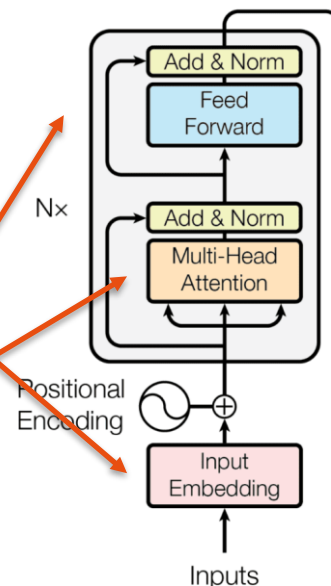
Which Weights are Updated during Training in BERT?

Backpropagation is the process of training Artificial Neural Networks (ANNs) by adjusting the weights of each layer in order to minimize a predefined error or cost function. Steps involved in backpropagation for pre-training and fine-tuning BERT are:

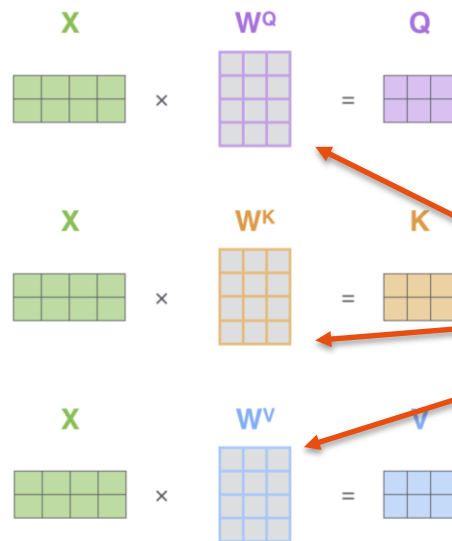
1. Feed forward: The input data is sent forward through the network, each layer performing some operation on the data. At the end of this process, an output is generated.
2. Calculate the error: The generated output is compared to the expected output (also known as the target) and an error is calculated.
3. Calculate the derivatives: The derivatives of the error with respect to the weights of each layer of the network is calculated.
4. Backpropagate the error: The derivatives are used to propagate the error backwards through the network. This helps in calculating the adjustments that need to be made to the weights in order to minimize the error.
5. Update the weights: The weights of each layer of the network are updated in the direction that helps to minimize the error.

The number of weights per layer depends on the model architecture. For example, BERT Base has 12 layers, each with its own set of weights:

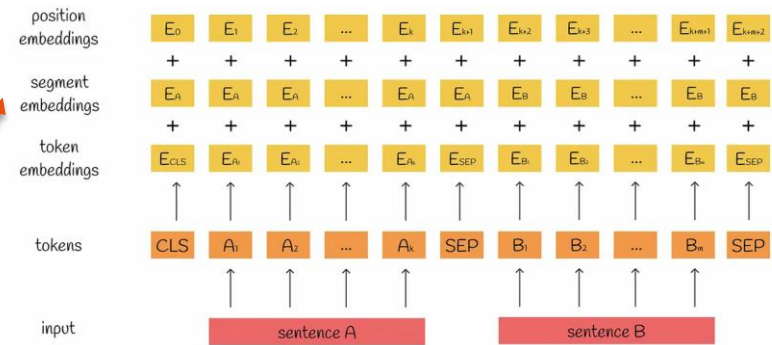
- The weights for the embeddings layer are also updated since this layer is a part of the network and is used to represent the input data.
- The weights for the Q, K, and V matrices are also updated as part of the backpropagation process.
- The feed forward neural network layer also has its own set of weights which are updated as part of the backpropagation process.



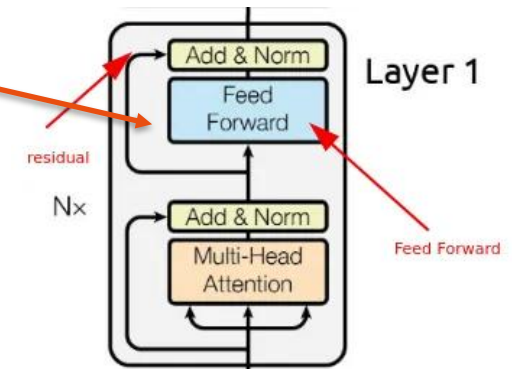
Weights that are updated in BERT



- The weights for the **embeddings layer** are also updated since this layer is a part of the network and is used to represent the input data.
- The weights for the **W^Q , W^K , and W^V matrices** are also updated as part of the backpropagation process for the proper self-attention calculations.
- The **feed forward neural network** layer also has its own set of weights which are updated as part of the backpropagation process.



How many trainable weights do you estimate that BERT base has: 12 layers, 768 dim embeddings and 12 attention heads.

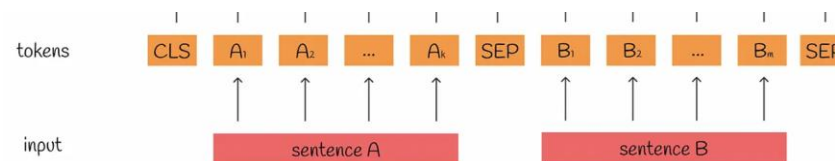


How many trainable Weights in BERT-base?

Let's start with the “Embedding Layer”

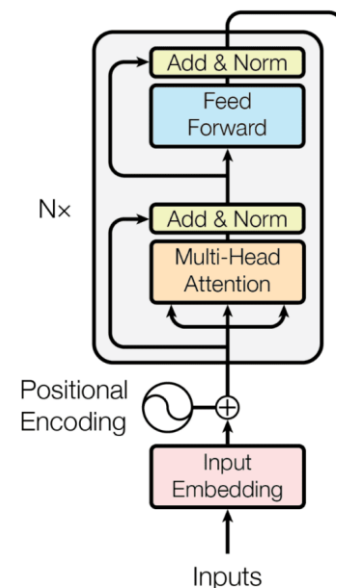
BERT uses a word embedding layer (vocabulary size: V) and positional embeddings.

- **Token Embeddings:** $V \times 768$
- **Positional Embeddings:** $P \times 768$ (usually $P=512$, the maximum sequence length)
- **Segment Embeddings:** 2 segments (A/B) $\times 768$



Embedding layer total:

$$\text{Embeddings} = V \times 768 + P \times 768 + 2 \times 768$$

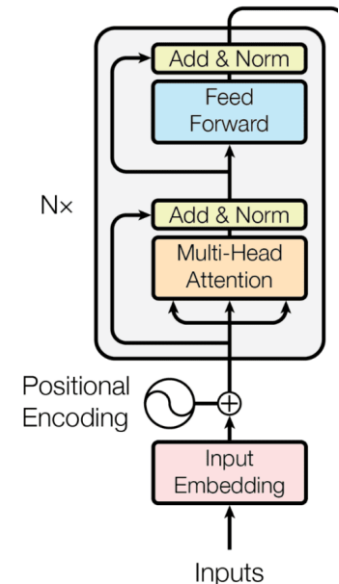


A closer look at the Embeddings during training

During training:

- **Token Embeddings:** These are updated. The model learns better token representations by optimizing these embeddings during pretraining tasks (Masked Language Modeling and Next Sentence Prediction). For example:
 - If "apple" is masked and predicted as "fruit," the corresponding embedding for "apple" is updated to better encode its contextual meaning.
- **Positional Embeddings:** These are also updated. While they encode fixed positional information initially, their representations can be fine-tuned to optimize performance on the training data.
- **Segment Embeddings:** These are updated as well. For tasks like Next Sentence Prediction or any task that uses segments, the model learns to adjust these embeddings to capture relationships between sentence pairs.

In summary, all components of the embedding layer are trainable and are updated during training.



. Why Not Just Update Segment Embeddings?

If only the segment embeddings were trainable:

- The model would fail to adapt the token embeddings to learn nuanced word meanings and their contexts.
- Positional embeddings would remain static, potentially limiting the model's ability to encode positional relationships effectively for different tasks.
- The model's performance on downstream tasks would be significantly worse.

Trainable Weights in BERT-base: 2. Transformer Layers 1/2:

There are 12 transformer layers, each with:

- **Self-Attention Weights:**

- Query, Key, Value projections (768×768 each) for each attention head:

$$Q/K/V = 3 \times 768 \times 768$$

- Output projection back to hidden size (768×768):

$$\text{Output Projection} = 768 \times 768$$

Total for self-attention:

$$\text{Self-Attention Layer} = (3 \times 768^2) + 768^2 = 4 \times 768^2$$

Trainable Weights in BERT-base: 2. Transformer Layers 2/2

- **Feedforward Layers:** Two fully connected layers in the feedforward network:

- Input to intermediate (768×3072):

$$\text{Intermediate FC} = 768 \times 3072$$

- Intermediate back to hidden size (3072×768):

$$\text{Output FC} = 3072 \times 768$$

Total for feedforward:

$$\text{Feedforward} = 768 \times 3072 + 3072 \times 768 = 2 \times 768 \times 3072$$

- **Layer Norms:** Two LayerNorm parameters per layer:

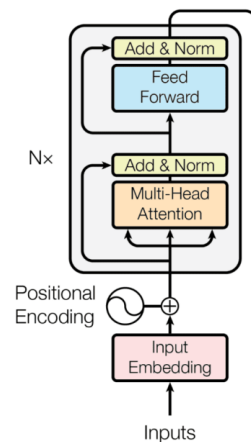
$$\text{LayerNorms} = 2 \times 768 \times 2 = 2 \times 768$$

Total per transformer layer:

$$\text{Per Layer Total} = (4 \times 768^2) + (2 \times 768 \times 3072) + (2 \times 768)$$

The **3072** comes from the fact that in BERT, the hidden size of 768 is expanded by a factor of **4** in the intermediate layer of the feedforward network. This design choice is based on the original Transformer architecture in the Vaswani et al. (2017) paper.

Two fully connected layers



Trainable Weights in BERT-base: Final Calculation:

Combine all layers and embeddings:

$$\text{Total Parameters} = \text{Embeddings} + 12 \times \text{Per Layer Total}$$

Assuming typical BERT-base settings:

- Vocabulary size (V) = 30,000
- Positional embeddings (P) = 512

Embeddings:

$$\text{Embeddings} = (30,000 \times 768) + (512 \times 768) + (2 \times 768) = 23,187,456$$

Per Layer:

$$\begin{aligned} \text{Per Layer Total} &= (4 \times 768^2) + (2 \times 768 \times 3072) + (2 \times 768) \\ &= (4 \times 589,824) + (2 \times 2,359,296) + 1,536 = 2,359,296 + 4,718,592 + 1,536 = 7,080,192 \end{aligned}$$

Total Parameters:

$$\text{Total Parameters} = 23,187,456 + (12 \times 7,080,192) = 23,187,456 + 84,962,304 = 108,109,760$$

~ 100.000.000 trainable weights!

Typical Pre-Training:

1. Dataset Size

The original BERT-base model was pretrained on a corpus containing **3.3 billion words**, combining:

- BooksCorpus (~800M words)
- English Wikipedia (~2.5B words)

2. Number of Training Steps

The original BERT-base pretraining was done for **1,000,000 steps** with:

- A batch size of **256 sequences**
- Sequence length **512 tokens**



See why 2 slides further...

This corresponds to processing about **40 epochs** over the dataset, assuming 3.3 billion words. The exact formula to estimate epochs is:

$$\text{Epochs} = \frac{\text{Steps} \times \text{Batch Size} \times \text{Sequence Length}}{\text{Total Tokens in Corpus}}$$

3. Recommended Epochs

While 40 epochs worked for the original BERT training, it might not always be necessary for other datasets or training setups. Some recommendations include:

- **Smaller Corpora:** Aim for 10-20 epochs for smaller datasets.
- **Large-scale Pretraining:** Expect 40-50 epochs for very large and diverse datasets.
- **Domain-specific Pretraining:** Focus on fewer epochs (e.g., 5-10) if you're fine-tuning BERT on domain-specific data like legal or medical text.

4. Compute Requirements

Pretraining for the full 1M steps took:

- **16 TPU v3 chips** (for ~4 days)
- Equivalent to several weeks on high-end GPUs like NVIDIA A100s.



For domain-specific adaptations (e.g., LegalBERT or BioBERT), you can often stop after **5-10 epochs** if you're starting from an existing pretrained checkpoint.

Why 1.000.000 training steps?

1. Dataset Size

BERT was pretrained on:

- **BooksCorpus:** ~800 million words.
- **English Wikipedia:** ~2.5 billion words.
- **Total words:** ~3.3 billion.

For tokenization, BERT uses WordPiece, splitting words into subword units. On average, one word corresponds to **1.1 to 1.2 tokens**. Therefore, the total number of tokens in the corpus is:

$$\text{Total Tokens} \approx 3.3 \text{ billion words} \times 1.1 = 3.63 \text{ billion tokens.}$$

2. Batch Size and Sequence Length

Each training step processes:

- **Batch size:** 256 sequences.
- **Sequence length:** 512 tokens.

This means each training step processes:

$$\text{Tokens per step} = 256 \times 512 = 131,072 \text{ tokens.}$$

3. Total Training Steps

To determine the number of steps, we calculate how many tokens need to be processed to cover the entire dataset for a single epoch:

$$\text{Tokens per epoch} = 3.63 \text{ billion tokens.}$$

With a batch size of 256 sequences and sequence length of 512, the number of steps per epoch is:

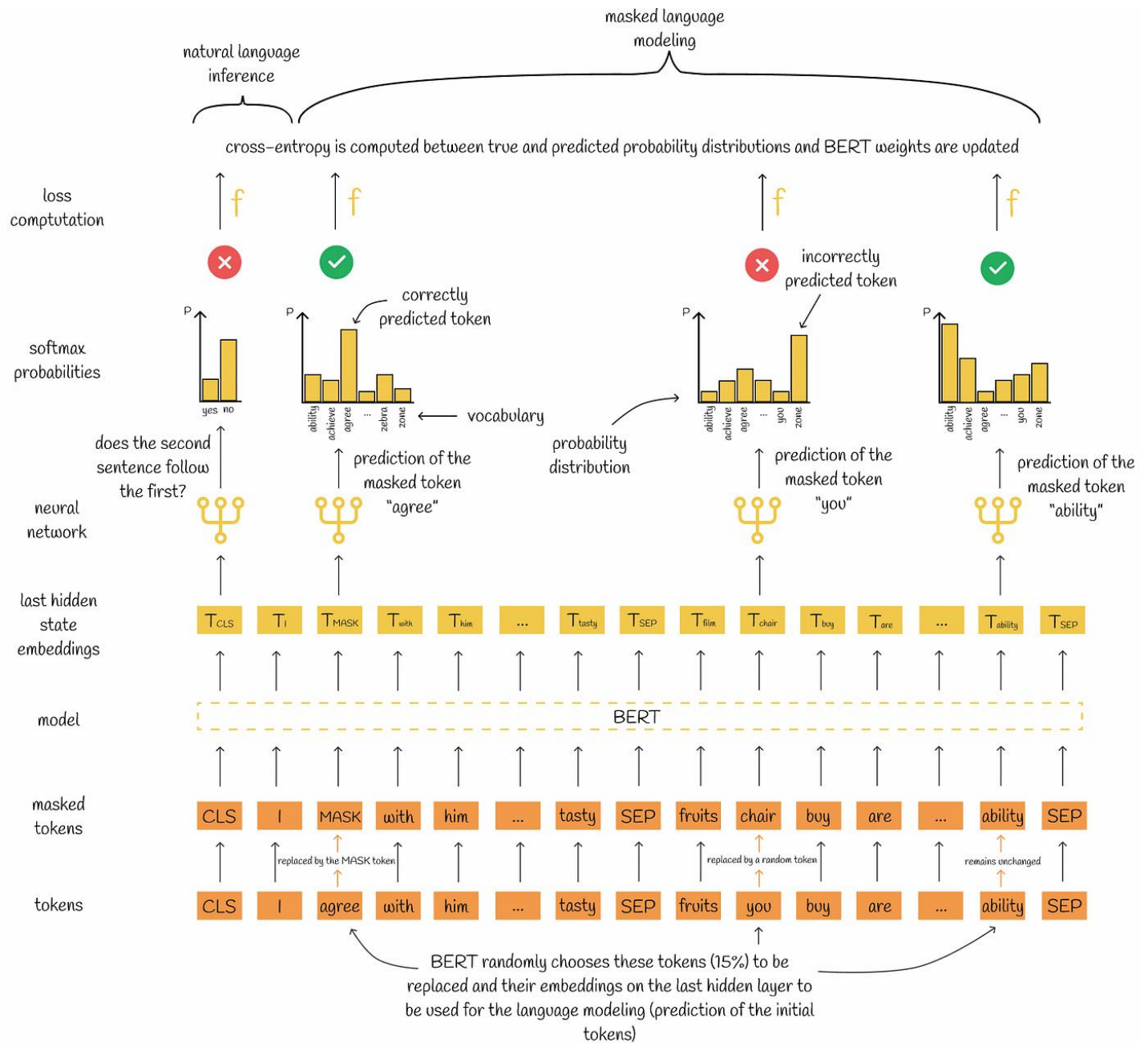
$$\text{Steps per epoch} = \frac{\text{Tokens per epoch}}{\text{Tokens per step}} = \frac{3.63 \times 10^9}{131,072} \approx 27,686.$$

The authors trained BERT for approximately **40 epochs** over the dataset, which gives:

$$\text{Total Steps} = 40 \times 27,686 \approx 1,107,440.$$

This is rounded to **1,000,000 steps**, which aligns with the computational budget and convergence observed in practice.

BERT: Pre-training for Natural Language Inference with Masked Language Modeling



BERT: After Pre-Training ...

- Once pre-training is completed, BERT can understand semantic meanings of words and construct embeddings which can almost fully represent their meanings.
- Fine-tuning will gradually modify BERT weights for solving a particular downstream task.
- BERT's ability to build bidirectional text representations is a major advantage dealing with linguistic phenomena such as homonyms and other ambiguous long-distance references (pronouns, co-references, ...).

Now, let's take a look at a number of fine-tuning operations for:

1. Sentence Pair Classification
2. Question-Answer Tasks
3. Single Sentence Classification
4. Single Sentence Tagging



Various BERT models

Model	Parameters (Millions)
BERT-Tiny	4.4
BERT-Mini	11.3
BERT-Small	29.1
BERT-Medium	41.7
BERT-Base	110.1
BERT-Large	340



Comparison	BERT October 11, 2018	RoBERTa July 26, 2019	DistilBERT October 2, 2019	ALBERT September 26, 2019
Parameters	Base: 110M Large: 340M	Base: 125 Large: 355	Base: 66	Base: 12M Large: 18M
Layers / Hidden Dimensions / Self-Attention Heads	Base: 12 / 768 / 12 Large: 24 / 1024 / 16	Base: 12 / 768 / 12 Large: 24 / 1024 / 16	Base: 6 / 768 / 12	Base: 12 / 768 / 12 Large: 24 / 1024 / 16
Training Time	Base: 8 x V100 x 12d Large: 280 x V100 x 1d	1024 x V100 x 1 day (4-5x more than BERT)	Base: 8 x V100 x 3.5d (4 times less than BERT)	[not given] Large: 1.7x faster
Performance	Outperforming SOTA in Oct 2018	88.5 on GLUE	97% of BERT-base's performance on GLUE	89.4 on GLUE
Pre-Training Data	BooksCorpus + English Wikipedia = 16 GB	BERT + CCNews + OpenWebText + Stories = 160 GB	BooksCorpus + English Wikipedia = 16 GB	BooksCorpus + English Wikipedia = 16 GB
Method	Bidirectional Transformer, MLM & NSP	BERT without NSP, Using Dynamic Masking	BERT Distillation	BERT with reduced parameters & SOP (not NSP)

Why Domain Dependent BERT Embeddings?

Bible:

The concept, “act of God,” is central to the biblical understanding of God and his relation to the world. Repeatedly we are told of the great works performed by God in behalf of his people and in execution of his own purposes in history.

“... an Act of God”

Legal Agreement:

e. Cancellations: Lyonpride is not liable for return of RMHB fees in the event of cancellations or other defaults by Lyonpride due to Acts of God. The RMHB does not have the right to terminate this Agreement for any delay or default in performing hereunder if such delay or default is caused by conditions beyond the control of Lyonpride including, but not limited to Acts of God.

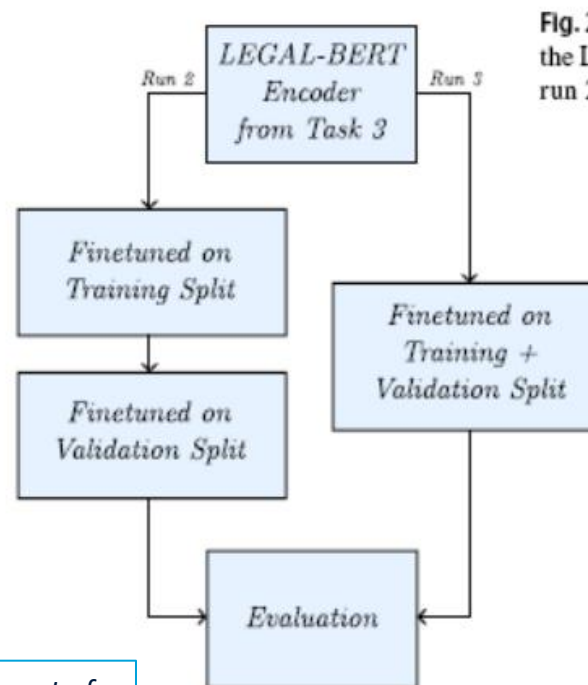


Fig. 2 Fine-tuning workflow of the LEGAL-BERT encoder for run 2 and 3

What to do with Domain Dependent Embeddings?

1. Use the embeddings in the fine-tuned BERT LLM model.
2. Extract the embeddings and use in T-5 (better translations), Pegasus (better summaries), GPT (better text generation), or other transformer-based model.

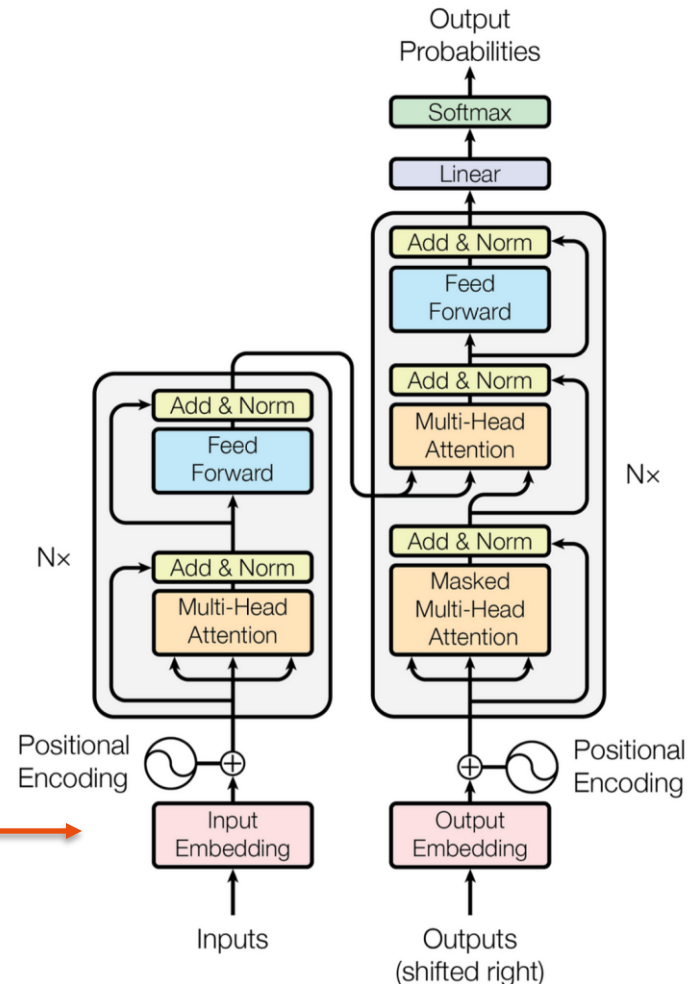
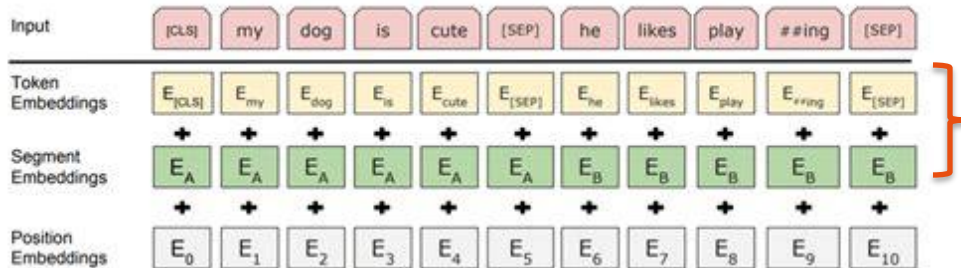


Figure 1: The Transformer - model architecture.

Using Adapters during Training



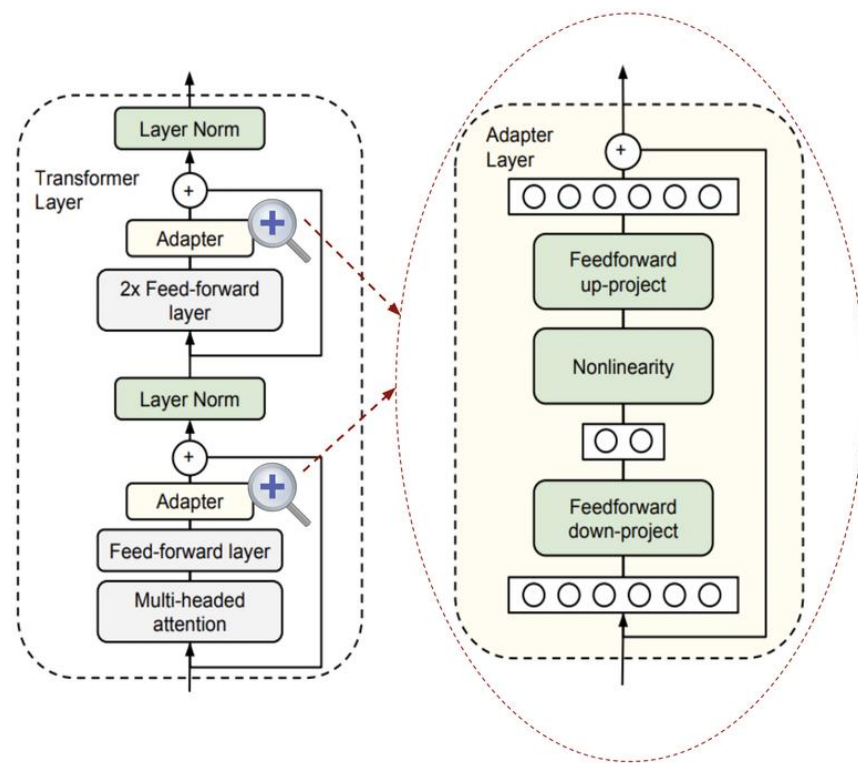
Maastricht University

What are Adapters?

Adapters are **small trainable modules** inserted into the layers of a pre-trained transformer model. Instead of fine-tuning all the parameters of a huge model (which can have hundreds of millions or even billions of parameters), adapters allow us to **freeze the original model weights** and only train these lightweight additional modules.

They were first introduced by Houlsby et al. (2019) in the paper "Parameter-Efficient Transfer Learning for NLP".

<https://arxiv.org/pdf/1902.00751>



How they Work

- A pre-trained transformer model (e.g. BERT) has many stacked layers (multi-head self-attention + feedforward blocks).
- Adapters are **bottleneck neural layers** inserted into each transformer block:
 - Input from the layer $\rightarrow$ goes into a **down-projection** (reduces dimensionality).
 - Then through a **non-linearity** (like ReLU or GELU).
 - Then an **up-projection** (back to the original hidden size).
 - Finally, the adapter output is **added back** to the main residual stream.

Formally, if h is the hidden representation from the transformer, then:

$$\text{Adapter}(h) = h + W_{up} f(W_{down}h)$$

where:

- $W_{down} \in \mathbb{R}^{d \times m}$ projects hidden dim d down to a smaller bottleneck m)
- f = non-linear activation
- $W_{up} \in \mathbb{R}^{m \times d}$ projects back up to original size)

Adapters are small, trainable “add-on” layers inside transformer blocks that enable parameter-efficient transfer learning.

Why use Adapters?

- Parameter efficiency: Only a few million parameters are added instead of fine-tuning the entire model.
- Multi-task learning: Different tasks can share the same frozen transformer backbone while each task has its own adapter.
- Storage efficiency: You don't need to save full copies of huge models, only the small adapter modules.
- Faster training: Much fewer parameters to update.

Example:

- Suppose you have BERT-base with ~110M parameters.
- A single adapter layer might add only ~2–4% extra parameters.
- For 10 different NLP tasks, instead of training 10 full BERTs, you train 10 small adapters (while sharing the main frozen model).

Think of the pre-trained model as a general-purpose language engine. Instead of reprogramming the whole engine for each task, you just plug in a small adapter chip that adjusts the outputs slightly for your specific task. This makes it possible to reuse the same big pre-trained model across many domains and tasks efficiently.

Reinforcement Learning



Maastricht University

Reasoning

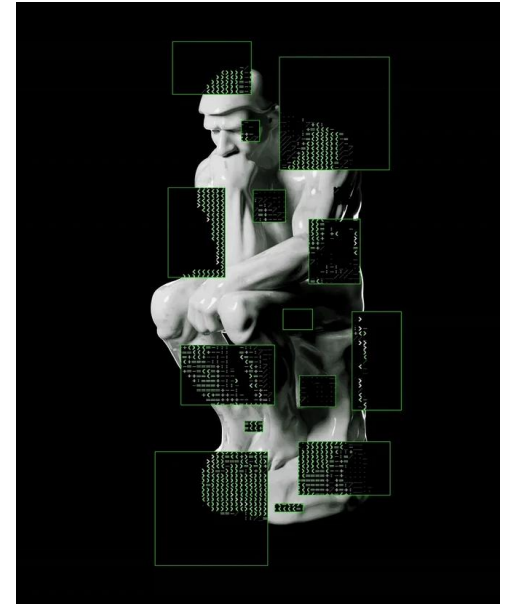


Maastricht University

The Strange Origin of AI's 'Reasoning' Abilities

- In 2020, 4chan gamers playing AI Dungeon discovered that asking GPT-3 to solve math "step by step" made it much more accurate. This was chain-of-thought prompting — over a year before Google researchers claimed to be first.
- The trick isn't magic. Extra steps just give the model more context, which steers its next-word prediction toward better answers — it learned this pattern from training text full of phrases like "wait, no" and "let me check."
- Apple's "The Illusion of Thinking" paper exposes the gap. Reasoning models can perform up to 65% worse when you add irrelevant details to a question, even if the actual facts stay the same.
- A model's reasoning steps and its final answer aren't always connected. It can show wrong steps and still land on the right answer.

Don't mistake a model explaining itself for proof it's actually reasoning. Test the system, don't trust the narration.



Any Questions on the Lecture

Now let's form groups of 4



Discuss Exercises Lecture

Questions?



Maastricht University

Exercise 1: Pre-Training Parameter Budget and Compute Scheduling (BERT-Base)

Review the parameter count derivation and pre-training regime for BERT-Base ($L = 12$ layers, hidden dimension $d = 768$, $h = 12$ attention heads, vocabulary size $V = 30,000$, and maximum sequence length $P = 512$).

- Calculate the parameter counts for the embedding layer components: Token Embeddings ($V \times d$), Positional Embeddings ($P \times d$), and Segment Embeddings ($2 \times d$). Explain why all three components must remain fully trainable rather than freezing the token embeddings.
- For a single Transformer layer, show how the parameter count is calculated across:
 - Multi-Head Self-Attention: Query, Key, Value projections ($3 \times d^2$) and the output projection (d^2).
 - Intermediate Feed-Forward Network: the expansion to $d_{\text{ff}} = 3072$ ($d \times 3072$) and projection back ($3072 \times d$).
 - LayerNorm parameters.
 - Combine these to verify that each layer contains approximately 7.08M weights, resulting in ~ 108.4 M total trainable parameters across 12 layers.
- Using the pre-training schedule from the lecture:

$$\text{Epochs} = \frac{\text{Steps} \times \text{Batch Size} \times \text{Sequence Length}}{\text{Total Tokens in Corpus}}$$

- Given a corpus of 3.63 billion tokens, a batch size of 256 sequences, and a sequence length of 512 tokens, calculate the number of steps required to complete 1 epoch and verify why 1,000,000 steps equates to approximately 40 epochs. Why are only 5–10 epochs recommended when performing domain-specific continual pre-training (e.g., BioBERT or LegalBERT)?



Exercise 2: Alignment, from PPO to DPO, GRPO and verifiable rewards

The lecture follows alignment from the three-stage recipe of InstructGPT (SFT, reward model, PPO) to the critic-free methods of 2024 and 2025.

- Step 1 (SFT): why was the checkpoint selected by reward-model score rather than by validation loss, although the validation loss started to rise after one epoch?
- Step 3 (PPO): how is the environment a bandit, what does the per-token KL penalty prevent, and why does PPO need four networks in memory?
- DPO: starting from the KL-regularised objective, explain in words how the reward model is eliminated. What does DPO lose compared with on-policy RL, and how does iterative DPO repair it?
- GRPO: what replaces the value model, how is the advantage of one sample computed, and why is this called grading on a curve? Name two of the 2025 fixes and the bias each repairs.
- RLVR: when can a checker replace the reward model, and what does that buy against reward hacking?

Exercise 3: Test-Time Reasoning, Reward Supervision and Verifiable Rewards

The lecture moves from prompting (Lecture 11) to models that learn to write their own chain of thought.

- Self-consistency as a Bayesian estimate: why is majority voting over sampled chains the maximum a posteriori answer, and what does it assume about the errors of individual chains?
- Explain the credit-assignment problem of an outcome reward, how a process reward model (Lightman et al. 2023) addresses it, and how best-of-N uses such a model at test time.
- DeepSeek-R1-Zero was trained with an outcome reward only. What emerged during training, what went wrong, and why did the authors drop process reward models and tree search?
- pass@k: an RL-trained model beats its base model at $k = 1$ but not at $k = 256$. Give the two readings of this result and the control experiment that separates them.
- Name the three ways to spend compute at test time and state when, according to Snell et al. (2024), a small model with more thinking beats a larger model.

Discussion



Maastricht University

On Process Supervision: Can a Model Learn to Reason Without Us Grading Every Step?

Process supervision scores every step of a solution; outcome supervision scores only the answer. Process supervision works better, and PRM800K needed 800,000 human labels on individual reasoning steps to show it.

If a model only reasons well where humans have graded the reasoning, has it learned to reason, or have we moved human supervision one level deeper into the model's thinking? Is there a point at which supervising the process is just doing the thinking ourselves?

Where does step-level supervision stop scaling, what can reinforcement learning with only verifiable outcomes offer in its place, and what is lost when the only thing that can be rewarded is being right at the end?

On Parrots and Maze Rats: Does Searching Over Guesses Amount to Thinking?

“There are now two axes on which we can scale: training time and run time, monkeys and years, parrots and rats.” For a decade, scaling meant bigger models on more text. Now a model is given time to think: sample many chains, search a tree, score the steps, pick the best.

The engine inside is still a next-token predictor. Does wrapping it in search and voting turn prediction into deduction, or is a rat in a maze still guessing, only more systematically?

Human reasoning also looks like search over proposals with a critic attached. If the architecture is the same, what, if anything, is the difference? And if there is none, why does it still feel like one?



On the Brain in a Jar: Does Attaching a World Give You a World Model?

A bare language model is a brain in a jar: text in, text out, no world, no body, no goal. An agent is that jar with things attached: a memory, tools, a planning loop. A 1979 chess program on an Atari still beats it, because the Atari keeps an exact board in memory and the model keeps a guess.

If the engine in the middle has no world model, does attaching a world (databases, APIs, sensors) give it one, or does it only give its guesses longer arms?

Is grounding something a system has, or something a system does? And if an agent acts in the world reliably without an internal model of it, does it matter that the model is missing?

Questions?



Maastricht University